UNIVERSIDAD POLITÉCNICA DE MADRID

E.T.S. DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO FIN DE GRADO

GRADO EN INGENIERÍA DEL SOFTWARE

Sistema de préstamos descentralizado basado en la red Blockchain de Ethereum

Desarrollado por: Aquilino Pascual Sanz

Dirigido por: Alberto Díaz Álvarez y Guillermo Sánchez Gutierrez-Cabello

Madrid, 17 de septiembre de 2026

Sistema de préstamos descentralizado basado en la red Blockchain de Ethereum
Desarrollado por: Aquilino Pascual Sanz
Dirigido por: Alberto Díaz Álvarez y Guillermo Sánchez Gutierrez-Cabello
Proyecto Fin de Grado, 17 de septiembre de 2026

E.T.S. de Ingeniería de Sistemas Informáticos
Campus Sur UPM, Carretera de Valencia (A-3), km. 7
28031, Madrid, España

Si deseas citar este trabajo, la entrada completa en BibTeX es la siguiente:

```
@mastersthesis{citekey,
    title = {Sistema de préstamos descentralizado basado en la red Blockchain
de Ethereum},
    type = {Bachelor's Thesis},
    author = {Pascual-Sanz, A.},
    school = {E.T.S. de Ingeniería de Sistemas Informáticos},
    year = {2026},
    month = {9},
}
```

Esta obra está bajo una licencia Creative Commons «Atribución-NoComercial-CompartirIgual 4.0 Internacional». Obra derivada de https://github.com/blazaid/UPM-Report-Template.



Todo cambio respecto a la obra original es responsabilidad exclusiva del presente autor.

Agradecimientos

A mis compañeros de universidad,
A los que me ayudaron, y a los que ayudé,
A los que me enseñaron, y a los que enseñé.
Lo mejor de toda la universidad.

Resumen

Este proyecto de fin de grado nace con el objetivo de desarrollar un sistema financiero descentralizado que ofrezca transparencia, seguridad y automatización en la gestión de préstamos. La motivación principal para realizar este proyecto radica en la necesidad de eliminar intermediarios innecesarios, reducir costos operativos y garantizar la trazabilidad de cada operación mediante contratos inteligentes (Smart Contracts) en una red Blockchain. Este enfoque brinda mayor confianza a los usuarios y permite una gestión eficiente de los préstamos **sin depender de entidades centralizadas.**

Se ha desarrollado una solución basada en Blockchain, Smart Contracts y Web3 para descentralizar el proceso de préstamos, garantizando transparencia y eficiencia. Mediante contratos inteligentes, se automatiza la concesión, gestión y penalización por impago, reduciendo errores humanos. Además, se permite la integración con oráculos lo que podría validar datos externos como tasas de interés o solvencia . El sistema opera dentro de una dApp financiera, donde los préstamos se registran en la Blockchain y se gestionan de forma segura mediante un contrato principal que administra los fondos, asegurando un servicio innovador, verificable y eficiente.

Palabras clave: Blockchain, Smart Contract, DeFi, Web3, Préstamos automatizados.

Abstract

This bachelor thesis has been created with the aim of developing a decentralized financial system that offers transparency, security and automation in loan management. The main motivation for this project lies in the need to eliminate unnecessary intermediaries, reduce operating costs and ensure the traceability of each transaction through smart contracts (Smart Contracts) in a Blockchain network. This approach provides greater confidence to users and enables efficient loan management without relying on centralized entities.

A solution based on Blockchain, Smart Contracts and Web3 has been developed to decentralize the loan process, guaranteeing transparency and efficiency. Through smart contracts, the granting, management and penalties for non-payment are automated, reducing human errors. In addition, integration with oracles is enabled, which could validate external data such as interest rates or creditworthiness. The system operates within a financial dApp, where loans are registered in the Blockchain and securely managed through a master contract that manages the funds, ensuring an innovative, verifiable and efficient service.

Keywords: Blockchain, Smart Contract, DeFi, Web3, Automated loans.

Índice general

1 Introducción **1**

1.1 Objetivos 2

1.2 Motivación 3

1.3 Estructura de la memoria 4

2 Estado de la cuestión **6**

2.1 ¿Por qué Blockchain y por qué Ethereum? 6

2.2 Tecnologías para el desarrollo 12

3 Arquitectura y Desarrollo **15**

3.1 Solución propuesta 15

3.2 Limitaciones del proyecto 15

3.3 Actores del sistema 18

3.4 Casos de uso 20

3.5 Entidades del sistema 22

3.6 Decisiones de implementación en el sistema 23

3.7 Diagrama Entidad - Relación de Smart Contracts 34

3.8 Descripción de Smart Contracts 35

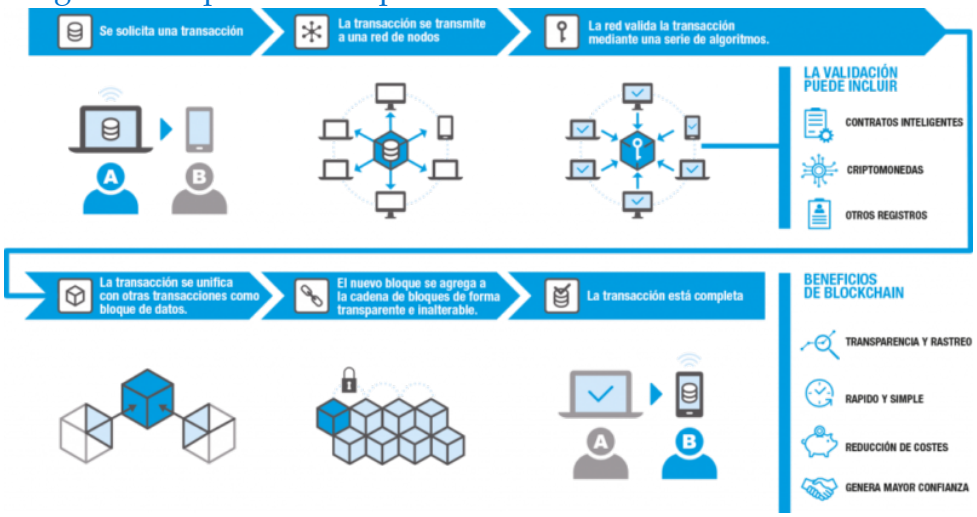
3.9 Validación de resultados 38

4 Conclusiones **45**

4.1 Impactos sociales y ambientales del desarrollo de este sistema a gran escala 46

Índice de figuras

2.1 Diagrama simplificado de proceso de transacción en en una blockchain.



Fuente: Neocheck[5] 9

2.2 Información a tiempo real del valor de criptomonedas, especialmente de *Solana* [8], *BNBChain* [9], *«Ethereum»* [7] las cuales sí permiten la integración de Smart Contracts. 10

3.1 Diagrama algoritmo semaforos que asegura el control de estados de los contratos del sistema. 16

3.2 Diagrama UML representando los actores del sistema relacionados con los casos de uso a los que tienen acceso. 19

3.3 Diagrama Secuencia que representa como la información vital del sistema queda registrada en la red aportando la inmutabilidad y confianza propia del proyecto 20

3.4 Diagrama entidad relación que representa la relaciones entre las entidades del sistema definiendo las funciones de este 28

3.5 Diagrama entidad relación que representa las relaciones y funciones entre los Smart contracts. 34

3.6 Ejemplo de login con Metmask 39

3.7 Información de contratos previa a la comprobación mensual en el recuadro amarillo y balance de la entidad financiera en el recuadro blanco. . . 41

3.8 Información de contratos posterior a la comprobación mensual en el recuadro amarillo y balance de la entidad financiera en el recuadro blanco. 42

3.9 Panel de control usuario. 43

3.10 Información Etherscan del registro en la red SepoliaETH del SmartContract principal 43

3.11 Registro en Sepolia en el que se puede ver una transacción o comprobante de pago Etherscan 44

4.1 Objetivos de desarrollo sostenible de la agenda europea para 2030 48

Índice de tablas

3.1 Casos de uso del sistema 21

3.2 Casos de uso para desarrollos posteriores 22

3.3 Casos de uso 1 23

3.4 Casos de uso 2 23

3.5 Casos de uso 3 24

3.6 Casos de uso 4 24

3.7 Casos de uso 5 25

3.8 Casos de uso 6 25

3.9 Casos de uso 7 25

3.10 Casos de uso 8 26

3.11 Casos de uso 9 26

3.12 Casos de uso 10 26

3.13 Casos de uso 11 27

3.14 Defínción de entidades del sistema 27

3.15 Descripción de atributos del Contrato Contract Factory 35

3.16 Descripción de funciones del Contrato Contract Factory 36

3.17 Descripción de atributos del Contrato Préstamo Contract 37

3.18 Descripción de funciones del Contrato Préstamo Contract 38

Índice de listados

3.1 Variable que contiene los prestamos producto 25

3.2 Variables contrato préstamo 28

3.3 Función para pagar mensualidad 28

r 29

3.4 struct TipoContrato 30

3.5 Función que devuelve los contratos de un usuario 31

3.6 Recuperacion de informacion de contratos 32

3.7 Muestra de informacion de contratos 32

1. Introducción

La tecnología blockchain ha transformado la forma en que entendemos las transacciones digitales, brindando un entorno seguro, transparente y, sobre todo, descentralizado, donde no es necesario depender de intermediarios financieros tradicionales y centralizados.

En un contexto económico global en el que algunos países enfrentan tasas de inflación elevadas, la adopción de monedas digitales como reserva de valor ha surgido como una alternativa atractiva para preservar el poder adquisitivo, intentando de algún modo evitar la erosión de valor que sufren las monedas locales.

Las criptomonedas ofrecen una opción descentralizada e internacional, proporcionando a los ciudadanos una alternativa que no está sujeta al control directo de los bancos centrales.

El aumento significativo de la inflación en algunas economías ha llevado a cuestionar el papel de los bancos centrales y sus políticas de control monetario, especialmente en lo que respecta a su influencia sobre la inflación y el valor de la moneda. Como respuesta a estas dinámicas, los criptoactivos han ganado popularidad en los últimos años como una forma de diversificación y preservación de valor en una economía global cada vez más conectada digitalmente.

La proliferación de proyectos DeFi refleja esta búsqueda de soluciones financieras alternativas que permiten a las personas interactuar directamente entre sí para obtener servicios financieros básicos, como préstamos y ahorros, sin intermediarios centralizados. Esta tendencia resalta el interés creciente por explorar formas de independencia económica, haciendo de las plataformas DeFi una propuesta relevante y de creciente adopción. A través de un sistema como el que se quiere proponer, que utiliza exclusivamente ETH y otros criptoactivos dentro de la red Ethereum, los usuarios podrían obtener préstamos respaldados por criptomonedas, sin la necesidad de instituciones financieras tradicionales y sin exponerse al impacto de decisiones de política monetaria centralizadas.

Este proyecto analiza si un sistema de préstamos descentralizado basado en Ethereum y

criptoactivos puede aportar mejoras respecto a los modelos tradicionales. A diferencia de estos, la descentralización permite reducir la intermediación, automatizar procesos mediante contratos inteligentes y ofrecer mayor transparencia en la gestión de los fondos.

Este trabajo pretende ofrecer una visión técnica y práctica de un sistema de préstamos DeFi, destacando sus oportunidades y desafíos, y evaluando su potencial para proporcionar una alternativa viable en el contexto financiero actual.

1.1. Objetivos

En esa sección se detallan los objetivos de este proyecto, estos se dividen en dos objetivos principales (Op) y distintos objetivos secundarios (Os) los cuales surgen en el desarrollo de los objetivos principales.

1.1.1. Objetivos principales

- **Op 1**: Desarrollo de un sistema de préstamos automatizado sin una entidad centralizada controladora, de acceso libre y cumpliendo los requisitos básicos de un proceso de prestamo como la seguridad, historial de cuotas, seguimiento, gestión de morosidad y personalización al usuario.
- **Op 2**: Desarrollar una interfaz unificada que permita a los usuarios, con o sin conocimientos técnicos, interactuar con el sistema de manera fácil y eficiente. La interfaz garantizará que todas las funcionalidades sean accesibles desde un solo punto, asegurando la correcta operación del sistema y facilitando la experiencia del usuario.

1.1.2. Objetivos secundarios

- **Os 1**: Explorar la viabilidad de los sistemas de identificación en aplicaciones DeFi.
- **Os 2**: Optimizar al máximo una solución en seguridad y eficiencia. Almacenando únicamente la información estrictamente necesaria para el funcionamiento correc-

to de la aplicación, evitando la exposición de datos sensibles , o la acumulación de información innecesaria en la red.

- **Os 3**: Optimizar eficiencia de ejecución de código en la red blockchain para reducir al máximo la cantidad de Gas necesario para el correcto funcionamiento del sistema.
- **Os 4**: Crear mecanismos de cumplimiento y penalización automatizado.
- **Os 5**: Explorar la flexibilidad y personalización de contratos inteligentes en productos financieros. Se investigarán maneras en las que el sistema sea capaz de ofrecer distintos productos financieros en función de la información que el sistema tenga del cliente como por ejemplo su situación financiera.
- **Os 6**: Optimizar la transparencia y privacidad en la consulta de información financiera.
- **Os 7**: Analizar cómo se puede gestionar la liquidez en sistemas DeFi y cómo se puede coordinar la transferencia de fondos en un sistema de préstamos en una red blockchain.

1.2. Motivación

Demostrar la viabilidad de un sistema de préstamos descentralizado mostrando cómo la tecnología Blockchain y los Smart Contract de Ethereum pueden ofrecer una alternativa sólida y segura frente a los sistemas financieros tradicionales, eliminando intermediarios y reduciendo costos y barreras de acceso.

Explorar y afrontar los desafíos técnicos específicos de desplegar un sistema DeFi en Ethereum esforzándose en conseguir gran seguridad y eficiencia, claves para la rentabilidad del proyecto los cuales se pueden convertir en puntos débiles de la red si no se realiza con detalle.

La motivación también radica en el valor social y económico que una solución como esta podría aportar. En un mundo en el que la inclusión financiera sigue siendo una meta distante para millones de personas, los sistemas de préstamos descentralizados tienen el potencial de ampliar el acceso a servicios financieros básicos, independientemente del lugar de residencia o del acceso a instituciones tradicionales.

Este proyecto responde a la creciente demanda de alternativas que no solo sean económicamente accesibles, sino también transparentes y seguras, proporcionando una herramienta que permita a los usuarios mantener una mayor autonomía sobre sus decisiones financieras en un entorno digital global. [1]

1.3. Estructura de la memoria

El código fuente de este proyecto se puede encontrar en el siguiente repositorio [2].

La estructura del documento es la siguiente:

- **Introducción:**
  - Presentación del problema y contexto económico de los préstamos descentralizados.
  - Motivación y relevancia del proyecto.
  - Objetivos generales y específicos del sistema.
- **Estado de la cuestión**:
  - Introducción a Blockchain y su importancia en el sector financiero.
  - Explicación de Ethereum y su ecosistema.
  - Tecnologías clave (Web3, Solidity, Remix IDE, Node.js, Truffle, etc.).
- **Diseño de la Arquitectura:**
  - Solución propuesta: Cómo se estructura el sistema de préstamos descentralizado.
  - Suposiciones y limitaciones del proyecto.
  - Actores del sistema y su interacción (Usuarios, Propietarios, Contratos).
  - Casos de uso principales y pantallas del sistema.
  - Diagramas UML, diagramas de secuencia y diagramas entidad-relación.
- **Diseño detallado:**
  - Explicación a nivel técnico de cada entidad del sistema.

- Smart Contracts implicados y su funcionamiento.
- Explicación detallada de la lógica del algoritmo principal.

- **Conclusiones y trabajo futuro**
  - Evaluación de los logros alcanzados en el proyecto.
  - Posibles mejoras y nuevas funcionalidades a explorar.

2. Estado de la cuestión

A continuación se definirán las tecnologías utilizadas en la solución ofrecida por este proyecto y también se definirán las claves por las que se han seleccionado dichas tecnología, desde la decisión inicial de utilizar una red blockchain hasta pequeñas decisiones de diseño como la selección del Framework para la UI

2.1. ¿Por qué Blockchain y por qué Ethereum?

2.1.1. Blockchain

Las redes blockchain representan una revolución en el mundo digital al proporcionar una infraestructura segura, descentralizada y transparente para la transferencia y almacenamiento de datos en forma de transacciones.

Las blockchain son bases de datos distribuidas (la información se replica en varios nodos) y consiste en una secuencia continua de bloques interconectados, donde cada bloque contiene una lista de transacciones y un código de referencia, o hash, que lo vincula al bloque anterior.

Éste hash es único, es decir, cualquier modificación en la información de los bloques utilizados para generar el hash modificaría el hash de referencia del bloque, de esta forma la red comprueba rápidamente si ha habido una modificación, (seguramente maliciosa) en la red. Esta estructura en cadena hace que la blockchain sea inmutable y resistente a manipulaciones, ya que cualquier cambio en un bloque alteraría todos los bloques posteriores, haciéndolo evidente para todos los participantes de la red.

Estos sistemas permiten la creación, registro y transferencia de información o valor sin necesidad de una entidad central de control. Su diseño y estructura presentan características innovadoras que la han posicionado como una de las tecnologías más prometedoras, sobre todo en el sector financiero, aunque su aplicación se está extendiendo a diversas industrias. [3]

Componentes en las transacciones Blockchain

Para comprender en profundidad el funcionamiento y la seguridad de una red blockchain, es fundamental analizar sus componentes clave, su estructura de datos, los mecanismos de consenso y las técnicas criptográficas que aseguran su integridad.

A través del siguiente paper: «*INTRODUCCIÓN A LA TECNOLOGÍA BLOCKCHAIN: SU IMPACTO EN LAS CIENCIAS ECONÓMICAS S*» [4] se ha reunido la información necesaria:

1. **Bloques**: La red Blockchain se estructura en bloques, que son conjuntos de información agrupada. Cada bloque contiene un conjunto de transacciones y un código de referencia al bloque anterior hash, formando una cadena continua. Los bloques también incluyen una marca de tiempo y un identificador único que garantiza su autenticidad.

2. **Nodos**: Son las computadoras que participan en la red. Cada nodo contiene una copia completa o parcial de la blockchain y es responsable de verificar y propagar transacciones y bloques. Los nodos se dividen en diferentes tipos, como nodos completos (que guardan la blockchain completa y participan en la verificación) y nodos ligeros (que solo descargan partes de la cadena y confían en nodos completos para la verificación).

3. **Mineros o validadores**: Son nodos que compiten para agregar nuevos bloques a la cadena realizando el proceso de validación de transacciones y siendo incentivados por su labor. Pueden ser:

    a) ***Proof of work* (PoW)**: Forma de validación en la que los nodos mineros compiten por resolver un problema matemático que implica encontrar un hash (código criptográfico) que cumpla ciertos requisitos establecidos por el protocolo. Dificil de resolver pero facil de verificar para otros nodos.

    b) ***Proof of Stake* (PoS)**: Los participantes de la red depositan una cantidad de criptomonedas como "participación.º "stake". Estos participantes se llaman validadores. Los validadores son seleccionados de forma pseudo-aleatoria para proponer y validar nuevos bloques, con probabilidades generalmente proporcionales a la cantidad de criptomonedas que tienen participando. Si el validador elegido valida correctamente recibe una recompensa, si actúa de manera malicionsa puede perder su participación.

4. **Hash criptográfico**: Cada bloque y cada transacción dentro de la blockchain está asegurada por un hash, que es un código único generado por los validadores. Este hash permite verificar de forma única el contenido del bloque y protege los datos de cualquier alteración. Si se modificara cualquier información en un bloque, el hash cambiaría, alertando al resto de la red de la alteración.
5. **Libro mayor distribuido**: La blockchain en conjunto actúa como un libro de contabilidad digital distribuido. Este libro mayor es inmutable, lo que significa que una vez registrada una transacción, no se puede modificar. Todos los nodos tienen acceso a la misma información, y las actualizaciones se sincronizan de forma continua y automática.

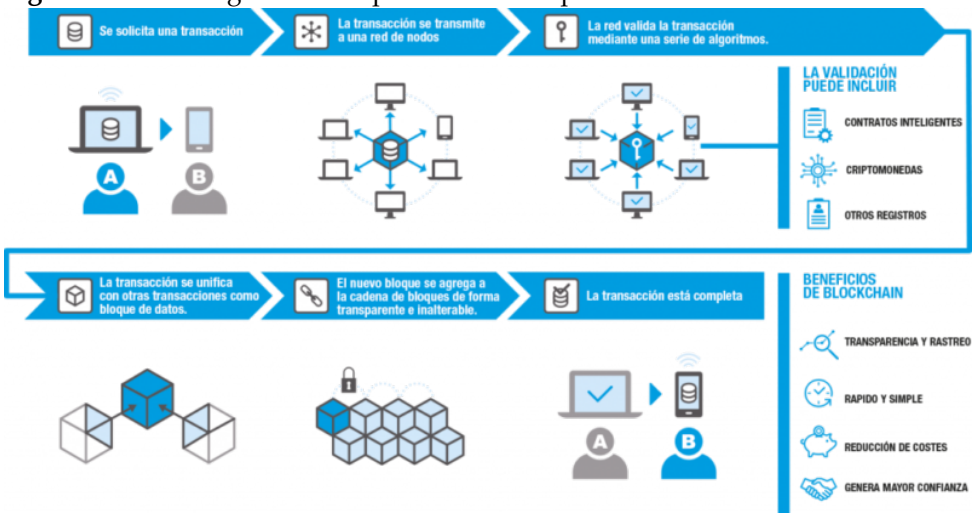
Proceso de transacción

Cada transacción en una blockchain sigue un proceso de verificación y registro antes de formar parte de la cadena:

1. **Creación de la transacción**: El usuario (o nodo emisor) crea una transacción con información como las direcciones del emisor y receptor, el monto de la transferencia y la firma digital generada con la clave privada del emisor.
2. **Difusión a la red**: La transacción es difundida a la red para su verificación. Los nodos recopilan las transacciones válidas y las agrupan en bloques para su posterior validación.
3. **Verificación de la transacción**: Los nodos validadores comprueban que la transacción es válida mediente Proof of Work, Prueba de trabajo (PoW) o Proof of stake, Prueba de participación (PoS).
4. **Agregación a la blockchain**: El bloque verificado se agrega a la cadena. Este proceso implica registrar la transacción inicial y todas las demas del bloque y vincular el nuevo bloque al bloque anterior mediante su hash. Esto asegura que la cadena sea inmutable y cronológica.
5. **Confirmación de la transacción**: Una vez que un bloque ha sido agregado, la transacción es confirmada. Las confirmaciones se refuerzan con cada bloque añadido posteriormente, haciendo la transacción prácticamente irreversible.

A través de la figura 1 se representa un diagrama simplificado del proceso de transacción.

Figura 2.1. Diagrama simplificado de proceso de transacción en en una blockchain.



Fuente: Neocheck[5]

Seguridad en Blockchain

La seguridad de la blockchain depende de varias capas de tecnología criptográfica y del diseño descentralizado de la red. Los elementos clave de su seguridad incluyen:

1. **Inmutabilidad**: Una vez que se agrega un bloque, es extremadamente difícil modificar los datos sin alterar el hash y afectar a todos los bloques posteriores. Cualquier intento de modificación sería rápidamente detectado por los nodos.

2. **Distribución y descentralización**: Al distribuir el control de la red entre miles de nodos (en el caso de las redes más importantes), blockchain elimina los puntos de fallo únicos. Esto significa que, incluso si un nodo es comprometido o hackeado, el resto de la red sigue siendo seguro, de hecho dependiendo de los criterios de cada red, realmente mientras ente el 51 y 70 % de nodos siga sin alterarse la red sigue siendo segura.

3. **Criptografía de clave pública y privada**: Cada usuario de la red tiene una clave pública y una clave privada. La clave pública funciona como una dirección visible para recibir activos, mientras que la clave privada permite firmar transacciones, asegurando la autenticidad de la actividad en la red. Solo el propietario de la clave privada puede autorizar transacciones en nombre de su dirección pública protegiendo los fondos.

4. **Hashing criptográfico**: El uso de algoritmos de hashing (como SHA-256 en Bitcoin) convierte los datos de las transacciones en códigos únicos de longitud fija.

Cualquier cambio en los datos originales generaría un hash completamente diferente, alertando a la red de la alteración. Este proceso es irreversible, lo que significa que el hash no puede descifrarse para revelar los datos originales.

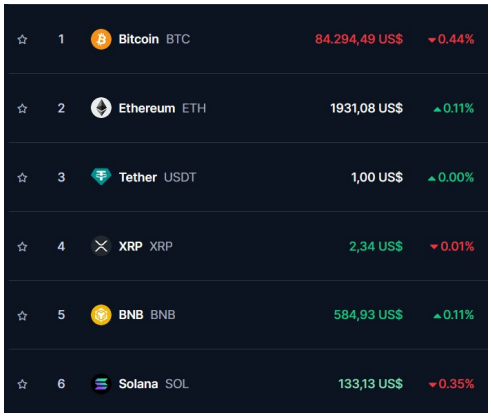
Con todas estas características se puede afirmar que una red blockchain es una herramienta segura para la realización de transacciones monetarias y de reserva de valor. Ahora bien, ¿cómo se puede crear un sistema automatizado?. En redes como Bitcoin, el único tipo de operación que se puede realizar es el de transacción. Este proyecto se apoya en una red con capacidad de ejecutar scripts o como son llamados dentro de la red: Smart Contracts. [6]

2.1.2. Ethereum

Ethereum es una plataforma de blockchain propuesta en 2013 por Vitalik Buterin a los 19 años de edad. [7] Se ha escogido esta red principalmente por su capacidad de despliegue de Smart Contracts, lo cual es necesario para este proyecto.

Aunque existen otras redes blockchain con capacidad de despliegue de Smart Contracts, como *Solana* [8] y *BNBChain* [9], al tratarse de un proyecto basado en la economía DeFi se ha seleccionado Ethereum debido a que su moneda es la que tiene mayor valor de mercado. Este se puede visualizar en la figura 2 a fecha de 14-03-2024 y a tiempo real en la web: CoinMarketCap[1]

Figura 2.2. Información a tiempo real del valor de criptomonedas, especialmente de *Solana* [8], *BNBChain* [9], «*Ethereum*» [7] las cuales sí permiten la integración de Smart Contracts.



Las principales características de Ethereum y en las cuales se apoya este proyecto son:

[1] https://coinmarketcap.com.

1. **Contratos Inteligentes y Máquina Virtual de Ethereum (Ethereum Virtual Machine, Máquina Virtual de Ethereum (EVM))** La característica más destacada de Ethereum es su capacidad para ejecutar Smart Contract. Estos son programas autoejecutables con condiciones predefinidas, que permiten automatizar una amplia variedad de procesos y acuerdos entre partes sin necesidad de intermediarios. La Máquina Virtual de Ethereum (EVM) es la infraestructura que hace posible la ejecución de estos contratos inteligentes esta consiste en un entorno virtual que permite que el código de los Smart Contract se ejecute por igual y de manera segura en todos los nodos de la red Ethereum, lo que garantiza consistencia e inmutabilidad. La EVM también es compatible con varios lenguajes de programación, siendo Solidity el más utilizado para desarrollar contratos inteligentes en la red.

2. **Plataforma para dApps** Ethereum fue la primera blockchain en introducir el concepto de aplicaciones descentralizadas (dApps). Las dApps son aplicaciones que funcionan en redes descentralizadas, permitiendo que los usuarios interactúen de manera directa y sin intermediarios en diferentes servicios, desde finanzas hasta juegos y redes sociales.

   Gracias a los contratos inteligentes y la EVM, Ethereum se convirtió en una plataforma que permite a los desarrolladores crear y desplegar sus propias aplicaciones sobre la misma red, similar a una "App Store"descentralizada. Esto le ha dado una gran popularidad y ha impulsado el crecimiento de sectores como las finanzas descentralizadas (DeFi) y los tokens no fungibles (NFT), lo que ha ampliado las posibilidades de uso de blockchain más allá de las transferencias de valor.

3. **Criptoeconomía y Modelo de Tokenización** Otra diferencia clave es que Ethereum cuenta con su propia criptomoneda, Ether (ETH), que es necesaria para realizar transacciones y ejecutar contratos inteligentes en la red. Los usuarios deben pagar 'Gas (una tarifa en ETH) para que sus transacciones y contratos inteligentes se procesen, lo que incentiva a los nodos mineros o nodos validadores (actualmente del tipo PoS)a verificar las transacciones y mantiene la red segura y funcional. Además, Ethereum introdujo el concepto de tokenización mediante el estándar 'ERC-20', que permite crear NFTs que representan activos o derechos sobre la misma red Ethereum. Este estándar ha facilitado la creación de una gran variedad de NFT y criptomonedas en Ethereum, impulsando el crecimiento de proyectos de DeFi y otros ecosistemas tokenizados. También existen otros estándares como ERC-721 y ERC-1155 para tokens no fungibles (NFT), que han revolucionado el sector de coleccionables y derechos digitales.

4. **Identidad** Ethereum es una red descentralizada, no utiliza identificaciones centralizadas como puede ser la centralización estatal del Documento Naciona de Identidad (DNI). Aunque no es una característica exclusiva de Ethereum. el sistema de identificación de esta red consiste en el típico sistema asimétrico de clave pública y privada de forma que para identificar a cada usuario se utiliza su clave pública y para que este pueda ejecutar transacciones en su nombre utiliza su clave única y privada. Sin embargo, esto también significa que no hay una forma nativa de verificar identidades del mundo real.

5. **Oráculos** son servicios externos que alimentan la blockchain con datos de fuentes externas, permitiendo que los contratos inteligentes interactúen con el mundo real. Por ejemplo, en aplicaciones DeFi relacionadas con bienes raíces, un oráculo podría verificar que una propiedad física corresponde a un token en Ethereum, permitiendo así transacciones con respaldo en activos físicos.

Se puede concluir que ethereum es un buen medio para desarrollar la intencionalidad del proyecto gracias a su EVM y la posiblidad de definir contratos inteligentes en ella, utilizando como moneda de cambio ETH e icluso añadir avales a los contratos y préstamos a través de los NFT y oráculos que los identifican en el mundo físico.

Sepholia ETH Sepolia[10]es una red de pruebas que permite a los desarrolladores experimentar y probar contratos inteligentes y aplicaciones descentralizadas en un entorno que simula la red principal de Ethereum pero sin utilizar Ether real. Para obtener SepoliaETH, los desarrolladores pueden utilizar faucets como Sepoliafaucet[11], que son servicios que proporcionan pequeñas cantidades de esta criptomoneda de prueba de forma gratuita.

2.2. Tecnologías para el desarrollo

2.2.1. Identificación en la red: Metamask

La naturaleza anónima de las redes Blockchain supone que se requiera una capa extra para relacionar un usario de la red a una persona física. Para la personalización de este proyecto se han buscado alternativas las cuales tengan librerías nativas capaces de interactuar directamente con las funciones de Web3 para una mayor personalización

del sistema. Encontramos distintas alternativas como Coinbase[12] y RabbyWallet[13] pero para este proyecto se ha decido hacer uso de Metamask[14] debido a su mayor popularidad y sobretodo la facilidad de acceso a nivel de usuario. *MetaMask – ¿Qué es MetaMask y qué ventajas tiene?* [15] Metamask es una extensión del navegador la cual actúa de 'cartera' de direcciones relacionando un usuario real con las direcciones que este tenga en las distintas redes disponibles. Al ser una extensión del navegador, puede detectar cuando el navegador se situa en una dApp y utilizarse para identificarse en la misma. A demás metmask también es un asistente para realizar transacciones sencillas de criptopmonedas.

2.2.2. Web3

Se le llama web3 a la próxima generación web (actual web2). Entre otras cosas buscará la descentralización y sobretodo el uso de dApp. A su vez web3 es un Framework de Javascript que permite acciones como conectarte a una red bockchain, gestionar cuentas, firmar transacciones, consultar saldos de las cuentas, interactuar con Smart Contracts. Este framework abstrae la complejidad de las interacciones con la blockchain, simplificando el desarrollo de dApps.

2.2.3. Solidity

Solidity[16] es un lenguaje de programación diseñado para escribir contratos inteligentes que se ejecutan en la blockchain de Ethereum. Es similar a JavaScript y C++, con características específicas para gestionar transacciones, almacenamiento en blockchain y lógica descentralizada. Es fundamental para desarrollar aplicaciones descentralizadas (dApps) y sistemas DeFi.

2.2.4. Remix IDE

Remix Project [17] es el conjunto de herramientas más exhausitivo que puede usarse para toda la experiencia de desarrollo de Smart Contracts para usuarios de cualquier nivel de conocimientos y como laboratorio de aprendizaje para enseñar y experimentar con Ethereum. También es deal como asistente de compilación y despliegue de Smart Contracts en Ethereum.

2.2.5. Node.js

Node.js [18]entorno de ejecución Javascript de código abierto y gratuito que facilita la incorporación de paquetes a un proyecto. Muy utilizado para instalar y utilizar librerías web3.

2.2.6. Mochatest Framework

Marco de pruebas en Javascript diseñado para ejecutar pruebas unitarias y de integración en aplicaciones Node.js. Ofrece una estructura flexible, con soporte para pruebas síncronas y asíncronas.

2.2.7. Truffle

Truffle [19] es un conjunto de herramientas para el desarrollo de aplicaciones en Ethereum. Principalmente puede compilar Smart Contracts y obtener información necesaria para su despliegue en la red. Truffle también tiene una herramienta muy poderosa: Ganache [20], esta es capaz de ejecutar una EVM en un entorno local. Perfecto para el desarrollo aunque con tendencia a fallos.

2.2.8. Explorador de bloques Etherscan

Etherscan[21] es un explorador de blockchain para Ethereum que permite visualizar transacciones, contratos inteligentes, balances de wallets y otra información en tiempo real. Es una herramienta clave para analizar y verificar actividades en la red Ethereum.

3. Arquitectura y Desarrollo

3.1. Solución propuesta

La propuesta que ofrece este proyecto para un sistema descentralizado basado en la red ethereum reúne todas las tecnologías anteriormente mencionadas creando una dApp con funciones básicas como la creación, ejecución y control de préstamos de cripto activos.

Este proyecto controla el pago de los préstamos en curso que pueda tener nuestro sistema cubre gracias a la inmutabilidad de la red junto con un algoritmo basado en semáforos: Cada préstamo tiene en cada momento un estado el cual cambia cuando se da la orden de **chequeo de mensualidad**, esta orden se produce mensualmente y transforma el estado de cada contrato dependiendo de si ha sido pagado o no.

El funcionamiento de este algoritmo se ve reflejado en la figura 3.1

3.2. Limitaciones del proyecto

3.2.1. Limitaciones legales

Durante este proceso de desarrollo, en diciembre de 2024, la Unión Europea aprueba el Reglamento Europeo de Criptoactivos [22], el objetivo de este reglamento es establecer normas para las entidades emisoras y proveedoras de servicios relacionados con la criptoactividad.

En el nuevo reglamento se tratan distintas cuestiones entre las que se encuentran:

1. Requisitos de transparencia absoluta a las plataformas de negociación de criptoactivos.

Figura 3.1. Diagrama algoritmo semaforos que asegura el control de estados de los contratos del sistema.

ESTADO 0	ESTADO 1	ESTADO 0
	 **CHEQUEO DE MENSUALIDAD**	
PLAZO ABIERTO	PLAZO ABIERTO (PENALIZACIÓN)	PLAZO ABIERTO
MES PAGADO	MES PAGADO (CONTINUA CON NORMALIDAD)	PLAZO ABIERTO
FINALIZADO	FINALIZADO (SE TANSFIEREN FONDOS A CONTRACT FACTORY Y SE BORRA DE MAPPING DE PRÉSTAMOS ACTIVOS)	FINALIZADO (ESPERANDO BORRADO)

2. Necesidad de autorización europea para lea emisión y provisionamiento de criptoactivos.
3. Protección del valor monetario en euros de los clientes.
4. Creación de medidas de control de las operaciones con información privilegiada y manipulación de mercado.

En esta normativa se especifican acciones obligatorias para entidades financieras que pueden adoptar este proyecto. Dentro de estas acciones se definen elementos comunes a cualquier empresa como definirse como persona jurídica o actuar de forma honesta, justa y profesional.

Las acciones que pueden llegar limitar el proyecto es la obligatoriedad de ofrecer la capacidad de convertir en Euros, los criptoactivos proporcionados por este proyecto. Específicamente se obliga a mantener:

1. Un fondo propio de 350.000E
2. El 2 % del importe medio de los activos de reserva
3. Una cuarta parte de los gastos generales fijos del año anterior
4. Establecimiento de planes de recuperación y reembolso en caso de no cumplir las obligaciones de la normativa

Estas acciones obligan a cualquier entidad que quiera hacer uso de este proyecto a aprovisionar las cantidades definidas en la normativa.

3.2.2. Limitaciones técnicas

El sistema desarrollado presenta una serie de limitaciones técnicas derivadas tanto del alcance definido para el proyecto como de las características propias de una solución basada en contratos inteligentes sobre la red Ethereum. Las principales limitaciones identificadas son las siguientes:

1. **Pagos anticipados:** el sistema no contempla actualmente la realización de pagos anticipados sobre los préstamos contratados.
2. **Cancelación anticipada:** no se contempla la posibilidad de cancelar un préstamo antes de que finalice el periodo establecido inicialmente en el momento de su contratación.
3. **Ausencia de garantías o colateral:** el sistema no incorpora mecanismos de colateralización que permitan garantizar el pago del préstamo en caso de impago. Esta funcionalidad se plantea como una posible ampliación futura mediante fondos bloqueados o activos tokenizados. Esta limitación restringe su aplicación en escenarios abiertos en los que no exista una relación de confianza entre las partes.
4. **Uso de un único activo:** el prototipo utiliza exclusivamente ETH como activo para la concesión y devolución de los préstamos. No se contempla actualmente el uso de otros tokens o activos digitales como unidad de préstamo, lo que limita la flexibilidad del sistema.

5. **Escalabilidad y consumo de Gas:** el incremento del número de préstamos activos implica un mayor número de operaciones que deben ser procesadas y registradas en la blockchain. En consecuencia, el coste acumulado de Gas puede aumentar a medida que crece el volumen de préstamos gestionados. Esta situación resulta especialmente relevante en las comprobaciones periódicas del estado de los préstamos, ya que una cantidad elevada de préstamos activos puede requerir un número significativo de operaciones y, por tanto, incrementar el coste de funcionamiento del sistema.

Estas limitaciones deben tenerse en cuenta al valorar una posible aplicación del sistema en un entorno real. La incorporación de mecanismos de colateralización, gestión de pagos anticipados, cancelación anticipada, soporte para múltiples activos y estrategias destinadas a reducir el coste de las transacciones constituyen posibles líneas de evolución futura.

3.3. Actores del sistema

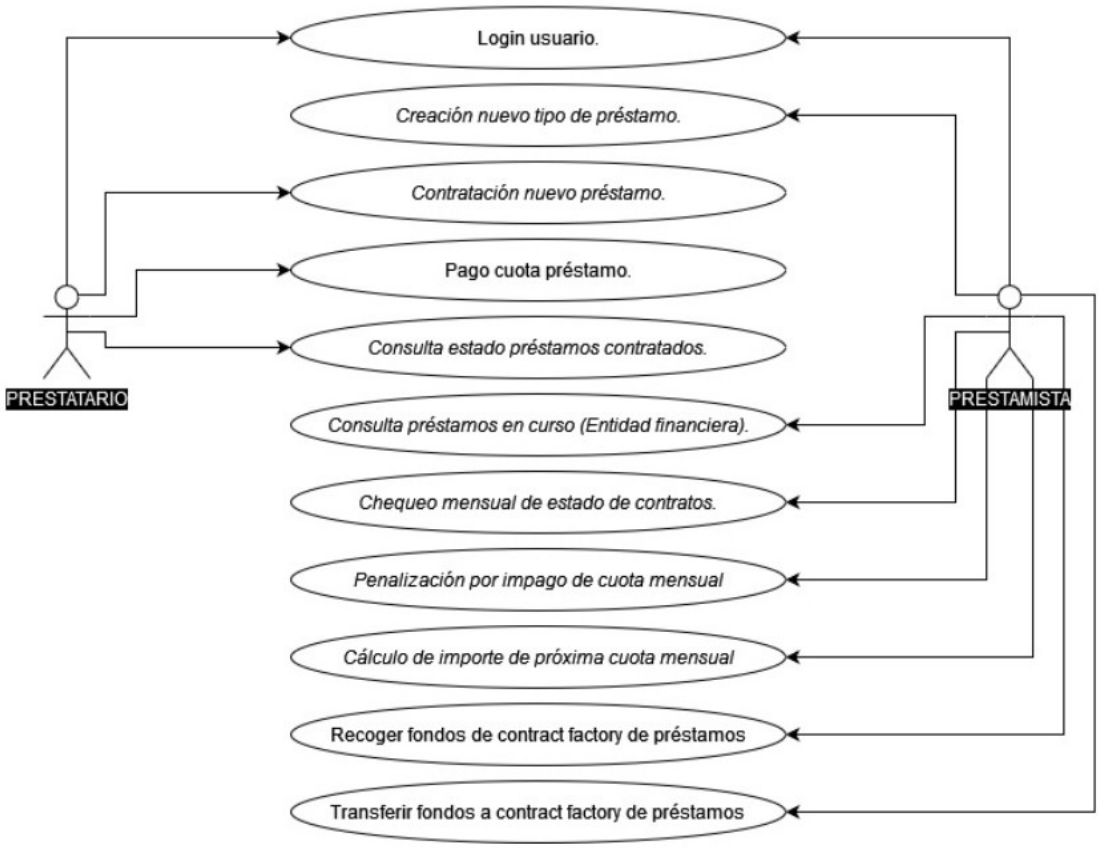
Los posibles actores que participan en el sistema son:

- **Usuario prestatario (User)** Usuario que accede al servicio de préstamos.
- **Entidad prestamista (Owner)** Entidad que ofrece los distintos tipos de préstamos.

3.3.1. Diagrama UML

Un diagrama UML (Unified Modeling Language) es una representación gráfica utilizada para modelar, visualizar, documentar y diseñar sistemas de software. En este caso relaciona los actores con los Casos de uso que pueden realizar y está representado en la figura 3.2.

Figura 3.2. Diagrama UML representando los actores del sistema relacionados con los casos de uso a los que tienen acceso.

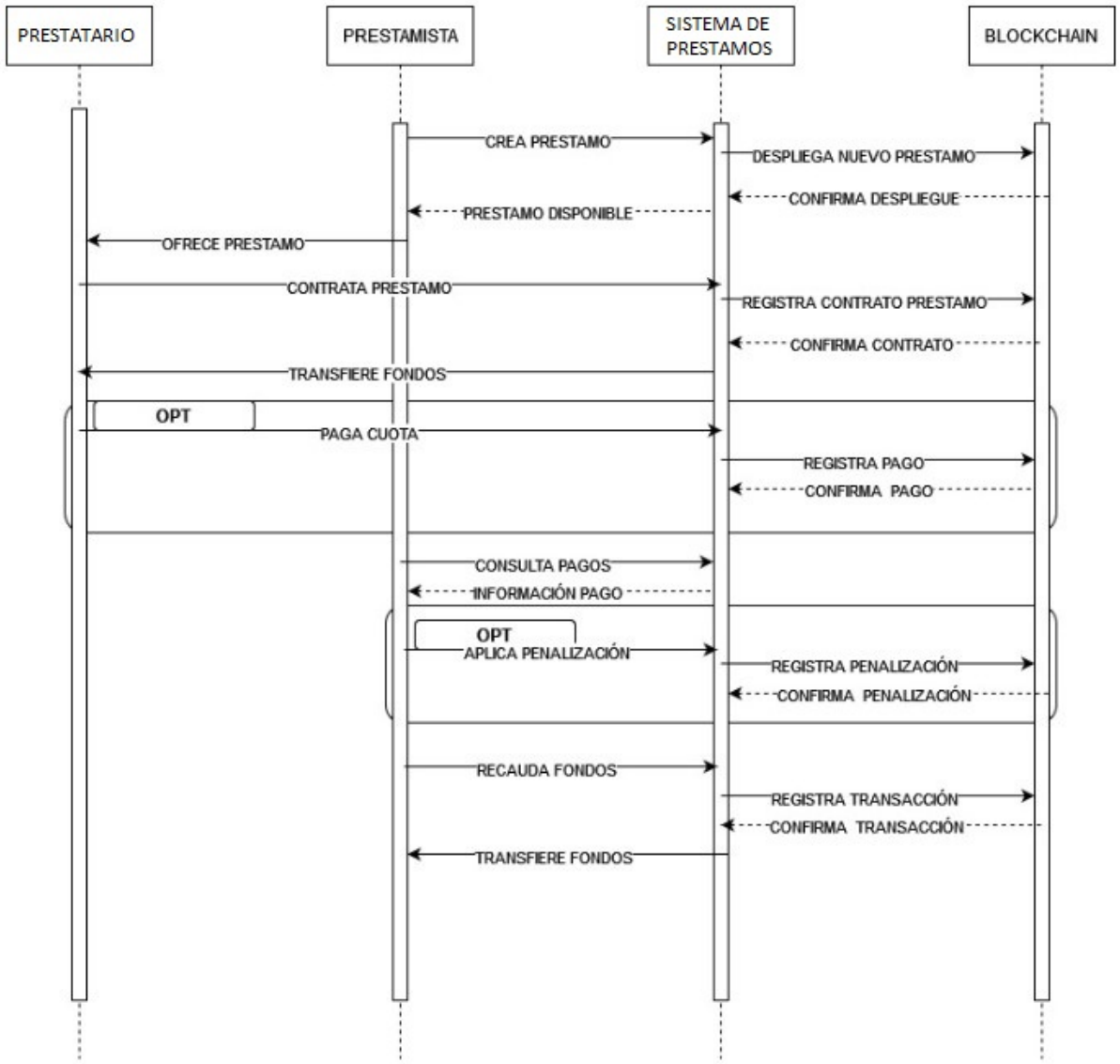


3.3.2. Diagrama de secuencia

Un diagrama de secuencia UML muestra cómo interactúan los objetos de un sistema en un orden temporal, representando el intercambio de mensajes entre ellos.

En este caso el diagrama de secuencia es de gran ayuda para comprender la cohesión del proyecto y la inmutabilidad de la información gracias a los registros en la blockchain tal y como se puede ver en la figura 3.3

Figura 3.3. Diagrama Secuencia que representa como la información vital del sistema queda registrada en la red aportando la inmutabilidad y confianza propia del proyecto



3.4. Casos de uso

En el marco del desarrollo del presente proyecto, se ha priorizado la definición de un conjunto de casos de uso que representan los escenarios esenciales para cumplir con los objetivos fundamentales del sistema ya que este proyecto debe de ser 'finito' y no 'infinito'.

La selección de los casos de uso no pretende abarcar una totalidad de posibles funcionalidades del sistema sino un conjunto operativo sólido y funcional. Comentando también un posible desarrollo futuro.

Estos casos de uso se definen en las tablas 3.1 y 3.2

3.4.1. Pantallas del sistema

Para un mayor entendimiento de los casos de uso se realiza una declaración las posibles pantallas que participan en el sistema son:

- **Pantalla principal común** Pantalla de incio comun a todos los usuarios, principalmente para hacer log in.
- **Pantalla principal de prestamista.**
- **Pantalla principal de prestatario.**

3.4.2. Casos de uso

Tabla 3.1. Casos de uso del sistema

Código	Descripción	Prioridad	Dependencia
CU1	Login usuario	10	
CU2	Creación nuevo tipo de préstamo	9	
CU3	Contratación nuevo préstamo	8	CU2
CU4	Consulta estado préstamos contratados	7	CU3
CU5	Pago cuota préstamo	6	CU3
CU6	Consulta préstamos en curso (Entidad financiera)	5	CU3
CU7	Chequeo mensual de estado de contratos	4	
CU8	Penalización por impago de cuota mensual	3	CU7
CU9	Cálculo de importe de próxima cuota mensual	2	CU3
CU10	Recoger fondos del prestatario	1	
CU11	Transferir fondos al prestamista	1	

3.4.3. Casos de uso para posteriores desarrollos

Tabla 3.2. Casos de uso para desarrollos posteriores

Código	Descripción
CU12	Garantía de dvolución en caso de impago
CU13	Consulta de valor de Euribor
CU14	Validación de solvencia económica de usuarios

- Para el caso de uso de la garantía se podría:
  - Bloquear fondos de otra cuenta a través de un Smart Contract la cual tuviera un balance mínimo
  - Bloquear un NFT dentro del Smart Contract tipo préstamo el cual se libera cuando se acabe la deuda.
  - Podría externalizarse esta responsabilidad a otra dApp
- La consulta del valor del Euribor se realizaría a través de un y serviría para el cálculo de la cuota mensual del préstamo.
- La validación de solvencia también se realizaría a través de un oráculo y se podría utlizar para personalizar los tipos de contrato al cliente.

3.4.4. Casos de uso detallados

3.5. Entidades del sistema

A continuación se definen las entidades cuyas relaciones compondrán y habilitarán las funciones del sistema propuesto

3.5.1. Diagrama entidad relación

El diagrama representado en la figura 3.4 pretende definir mediante relaciones entre las entidades las funciones que definen el sistema.

Tabla 3.3. Casos de uso 1

CU1	Log in usuario.
Descripción	Los usuarios accederán al sistema a través de Metamask
Pantalla	Pantalla inicial. (Se redirecciona a esta página si no se ha realizado el log in)
Rol	Cualquier usuario
Flujo	- El usuario accede a nuestra página web está le pedirá autenticarse con Metamask para poder acceder al servicio - La extensión de Metamask permite la autenticación del usuario y la página se redirige a la respectiva página principal dependiendo de si es la entidad prestamista o no.

Tabla 3.4. Casos de uso 2

CU2	Creación nuevo tipo de préstamo.
Descripción	La entidad prestamista puede crear nuevos productos tipo "préstamo"
Pantalla	Pantalla principal de la entidad prestamista.
Rol	Owner
Flujo	- El usuario owner tiene un formulario en su página principal para rellenar la información necesaria para la creación de un nuevo tipo de préstamo. - Este contrato se le mostrará al usuario prestatario en su página principal

3.6. Decisiones de implementación en el sistema

En esta sección se define el diseño de implementación utilizado para los smart contract, así como los atributos(Tablas 3.17 y 3.19) y funciones(Tablas 3.18 y 3.20) de cada uno de ellos.

Tabla 3.5. Casos de uso 3

CU3	Contratación nuevo préstamo.
Descripción	Los usuarios prestatarios pueden seleccionar el préstamo que desean contratar
Pantalla	Pantalla principal de usuarios prestatarios.
Rol	User
Flujo	- En la página principal el usuario podrá visualizar los posibles préstamos que puede contratar. - El usuario contrata un nuevo préstamo pinchando en el botón de "Contratar". Tan pronto como se procese la transacción, web 3 llamará al smart contract correspondiente y generará otro contrato el cual reflejará el estado del prestamo en cada momento. También las unidades solicitadas en el préstamo serán transferidas automáticamente desde los fondos del Smart Contract de préstamos a la dirección del usuario.

Tabla 3.6. Casos de uso 4

CU4	Consulta estado préstamos contratados
Descripción	Los usuarios del servicio podrán consultar el estado de sus préstamos contratados visualizándolos en su página principal
Pantalla	Pantalla principal usuario
Rol	User
Flujo	En el momento que el usuario hace log in ya puede visualizar los contratos que tiene contratados

3.6.1. Contract Factory

El lenguaje utilizado para el desarrollo de los Smart Contracts es Solidity El patrón de diseño en el que se basa el desarrollo de los Smart Contracts de este proyecto se llama *'Contract Factory'*. Este patrón de diseño encaja perfectamente en el desarrollo de este proyecto ya que nos permite crear los distintos contratos de préstamo de forma dinámica y automática. En esta arquitectura, un contrato principal (el *Factory*) actúa como intermediario recibiendo las solicitudes y generando nuevos contratos (*contratos préstamo*) según las necesidades de los usuarios o la aplicación. Dentro de este contrato se definen los distintos tipos de préstamos que pueden ser contratados por los usuarios,

Tabla 3.7. Casos de uso 5

CU5	Pago cuota préstamo.
Descripción	Los usuarios del servicio de préstamo realizan el pago de sus mensualidades a través de la DApp del proyecto
Pantalla	Pantalla principal de usuarios prestatarios
Rol	User
Flujo	-En la página principal del usuario aparecen los distintos prestamos que tiene contratados y su respectivos estados, junto a esto, existe un botón para cada contrato que transfiere la mensualidad calculada previamente desde la cuenta del usuario a la cuenta del Smart Contract de préstamo.

Tabla 3.8. Casos de uso 6

CU6	Consulta préstamos en curso (Entidad financiera).
Descripción	La entidad financiera puede consultar el estado de cada préstamo en curso
Pantalla	Pantalla principal de la entidad financiera
Rol	Owner
Flujo	-En la página principal del propietario aparecen los contratos en curso

Tabla 3.9. Casos de uso 7

CU7	Chequeo mensual de estado de contratos.
Descripción	La entidad financiera ejecuta la orden de comprobar que mensualidades no han sido pagadas ese mes.
Pantalla	Pantalla principal de entidad financiera
Rol	Owner
Flujo	Desde la página principal del propietario se tiene acceso al botón que lanza esa orden siempre y cuando hayan pasado 30 días desde la última orden.

asi como las distintas funcionalidades de gestión y monitorización. Para ello este contrato cuenta con varibles como:

Listado 3.1. Variable que contiene los prestamos producto

```
1    mapping(address => PrestamoCursando[])  prestamosCursando;
```

Tabla 3.10. Casos de uso 8

CU8	Penalización por impago de cuota mensual
Descripción	Los contratos en estado de impago generan una penalización de deuda, esta penalización solo puede suceder una vez por cada chequeo mensual de contratos
Pantalla	Pantalla principal de entidad financiera
Rol	Owner
Flujo	Cuando el propietario genera la orden de chequeo mensual los contratos que tengan estado de "impago" aumentaran su valor de deuda

Tabla 3.11. Casos de uso 9

CU9	Cálculo de importe de próxima cuota mensua
Descripción	El importe de la próxima cuota mensual es un valor dinámico sujeto a penalizaciones
Pantalla	Pantalla principal usuario
Rol	User
Flujo	- Tras el chequeo el mensual la cuota del próximo pago se podrá ver modificada en caso de penalización - En la información de los préstamos contratados aparecerá la cantidad de la cuota del mes en curso

Tabla 3.12. Casos de uso 10

CU10	Recoger fondos del contrato de prestamos
Descripción	La entidad bancaria propietaria del sistema de prestámos puede disponer de los fondos almacenados en el sistmea cuando desee
Pantalla	Pantalla principal de Owner
Rol	Owner
Flujo	La entidad bancarai tendrá una casilla en su página principal para escribir y recibir el importe desde la dirección del contract factory a su dirección de la red

```
2     address[] clientes;
```

Por otra parte, cada *contrato préstamo* mantiene información específica como el prestatario, la deuda restante, el número de cuotas pendientes, el estado del préstamo y el

Tabla 3.13. Casos de uso 11

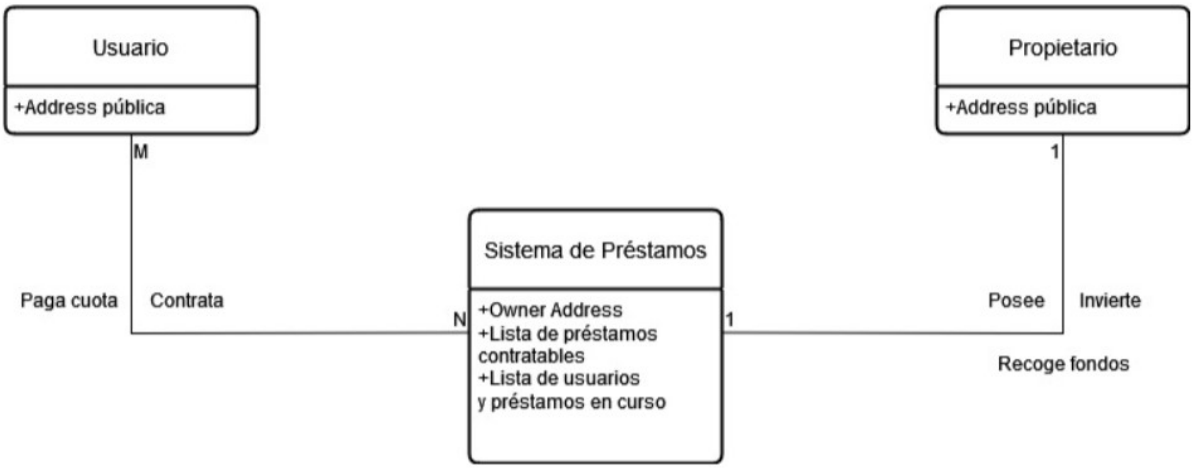
CU11	Transferir fondos al sistema de préstamos
Descripción	La entidad bancaria puede transferir cualquier cantidad de unidades al sistema de prestamos para mantener un saldo positivo y poder realizar estos préstamos
Pantalla	Pantalla principal Owner
Rol	Owner
Flujo	La entidad tendrá una casilla en su página principal para escribir y transferir el importe deseado a la dirección del sistema de préstamos.

Tabla 3.14. Defincón de entidades del sistema

1	Usuario.
Descripción	Usuario que accede al servicio de préstamos.
Atributos	Dirección pública de la red.
Acciones	- Pedir préstamo - Pagar préstamo
2	Propietario.
Descripción	Propietario del sistema, único capaz de recolectar los fondos del sistema. Normalmente una entidad financiera
Atributos	Dirección pública de la red.
Acciones	- Transferir unidades al sistema - Recoger unidades del sistema
3	Sistema de préstamos
Descripción	Conjunto de smart contract que definen los elementos del sistema y cuya ejecución valida las transacciones del sistema
Atributos	- Dirección de propietario - Lista de préstamos contratables - Lista de usuarios y sus préstamos en curso
Acciones	- Ejecutar las transacciones ordenadas por los usuarios dle sistema - Registrar en la red blockchain la informacion del sistema.

porcentaje de penalización aplicable como podemos comprobar en la definición de las variables de este prestamo en 3.2

Figura 3.4. Diagrama entidad relación que representa la relaciones entre las entidades del sistema definiendo las funciones de este



Listado 3.2. Variables contrato préstamo

```
        uint index;
        enum State { PAGADO, ABIERTO, FINALIZADO }
        State state;
        address prestatario;
        address entidadFinanciera;
        address payable prestamosFactory;
        uint256 cuotasRestantes;
        uint256 cantidadPrestada;
        uint256 deudaRestante;
        uint256 cuotaMensual;
        uint256 penalizacion;
        uint256 ultimoCheckeo;
```

Al contar cada contrato préstamo con su propia información, se garantiza que cada préstamo es independiente y puede ser gestionado de manera autónoma, mientras que el contrato factory centraliza la creación y supervisión de todos los préstamos. Tal y como se puede ver en la función 3.3 del *contrato préstamo*

Listado 3.3. Función para pagar mensualidad

```
        function pagarMensualidad() payable public {
```

```
                require(msg.sender == prestatario, "Only borrower");
                require(msg.value == cuotaMensual, "Send exact debt
                   value");
                require(state == State.ABIERTO, "Payment period not
                   open");

                state = State.PAGADO;
                deudaRestante -= cuotaMensual;
                cuotasRestantes--;


                if(cuotasRestantes == 1){
                    cuotaMensual = calculoCuotaMensual();
                }

                if(cuotasRestantes == 0){
                state = State.FINALIZADO;
                }
                transferirFondosAFactory(msg.value);

            }
```

A través de las lineas

```

Para evitar la creación de varios tipos de \textit{contrato prestamo}
    para distintos tipos de préstamo, se ha optado por crear un único
    contrato que pueda ser parametrizado en el momento de su creación
   .
En el contrato \textit{factory} se define un \textit{struct} que
   contiene los parámetros de cada tipo de préstamo, como el importe,
    los intereses, la penalización por impago y el número de cuotas.
            \begin{lstlisting}[language=Solidity, caption={struct
               TipoContrato},label={lst:structtipocontrato}]
                struct TipoContrato {
                    uint256 index;
                    uint256 porcentajeInteres;
                    uint256 cantidad;
                    uint256 deudaTotal;
```

```
10                  uint256 cuotas;
11                  uint256 penalizacionImpago;
12              }
```

Cuando un usuario solicita un préstamo, se pasa el tipo de préstamo seleccionado al constructor del contrato préstamo 3.4, que inicializa sus variables con los valores correspondientes.

Listado 3.4. struct TipoContrato

```
1               constructor(uint256 _index, address _prestatario,
                   address _entidad, uint256 _cuotasRestantes,
                   uint256 _cantidad, uint256 _deudaTotal, uint256
                   _penalizacion){
2                   index = _index;
3                   state = State.ABIERTO;
4                   prestatario = _prestatario;
5                   entidadFinanciera = _entidad;
6                   prestamosFactory = payable(msg.sender);
7                   cuotasRestantes = _cuotasRestantes;
8                   cantidadPrestada = _cantidad;
9                   deudaRestante = _deudaTotal;
10                  penalizacion = _penalizacion;
11                  cuotaMensual = calculoCuotaMensual();
12                  ultimoCheckeo = block.timestamp;
13              }
```

3.6.2. Direcciones como identificación

En un sistema basado en Ethereum no es necesario generar un identificador centralizado para identificar a un usuario. La dirección pública de su cartera constituye un identificador único dentro de la red. Cada solicitud de producto solicitada al Smart Contract factory genera un nuevo contrato préstamo en la red con su correspondiente dirección en ella quedando registrado en la variable *prestamosCursando*, Esta variable relaciona la dirección del usuario solicitante con un array de direcciones de los contratos

de préstamo que tiene en curso. De esta forma se puede registrar las direcciones de los contratos creados, proporcionando un mecanismo centralizado para su administración, por ejemplo a través de la función 3.6

Listado 3.5. Función que devuelve los contratos de un usuario

```
1    function verContratos(address _prestatario) public view returns
        (PrestamoCursando[] memory){
2        require((msg.sender == owner)||(msg.sender == _prestatario),
            "Top Secret");
3        return (prestamosCursando[_prestatario]);
4    }
```

3.6.3. Utilización de la plataforma Metamaskenta de identificación en el sistema

Debido a que la identificación de los usuarios en la red se realiza a través de sus direcciones públicas, es necesario que los usuarios dispongan de una cartera digital que les permita interactuar con la red Ethereum de una manera mas amigable. Para ello se ha optado por utilizar la extensión de navegador Metamask a los usuarios hacer login en la plataforma con una simple extensión del navegador, así como firmar transacciones de manera segura. El uso de Metamask hace que la aplicación no se haga responsable de la generación, almacenamiento ni gestión de las claves privadas de los usuarios.

3.6.4. Representación de cantidades monetarias

Debido a que la versión estable del lenguaje *solidity* para los Smart Contractsen el momento del desarrollo del proyecto puede tener ciertos errores a la hora de tratar decimales, la mejor decisión es tratar todas las cifras monetarias como enteros de la únidad mínima (Wei) evitando así los distintos problemas que puedan surgir con los decimales

3.6.5. Reducción del consumo de gas

Siempre que una operación no requiere necesariamente modificar el estado de la blockchain, esta se ejecuta en la aplicación web del cliente. De esta manera se evita utilizar Gas para operaciones puramente relacionadas con la visualización de información, como el recorrido de los contratos de un usuario para mostrar su estado. En este sistema se recupera la información de todos los contratos y es la aplicación web la que filtra y muestra la información de cada contrato, tal y como se puede ver en las siguientes lineas de código 3.6 y 3.7, la primera recupera la infromación de la red y la segunda muestra por pantalla

Listado 3.6. Recuperacion de informacion de contratos

```
        async iniciarVistaContratados(){
    this.direccionesContratados = await this.factoryContract.methods
        .verContratos(localStorage.getItem('userAddress'))
            .call({ from: localStorage.getItem('userAddress')});

        this.direccionesContratados.forEach(async (direccion: any) =>
            {
        var prestamoCursando = new this.web3.eth.Contract(
           PRESTAMOCURSANDO.abi, direccion);
        var contratado = await prestamoCursando.methods.mostrarInfo()
           .call({from: localStorage.getItem('userAddress')});
        contratado._ultimoCheckeo = this.secondsToDateString(
           contratado._ultimoCheckeo);
        contratado._state = this.contratoStateToString(contratado.
           _state);
        this.listaContratados.push(contratado);
        });
 }
```

Listado 3.7. Muestra de informacion de contratos

```
<ion-col *ngFor="let prestamo of listaContratados;" size="3.1"  >
     <ion-card >
       <ion-card-content >
```

```
4           <ion-list >
5
6             <ion-item >
7               <ion-label  class = "ion-text-center" >Estado: {{
                    prestamo._state.toString() }}</ion-label>
8             </ion-item>
9
10            <ion-item >
11              <ion-label  class = "ion-text-center" >Deuda Restante:
                    {{ prestamo._deudaRestante.toString() }}</ion-label>
12            </ion-item>
13
14            <ion-item >
15              <ion-label class = "ion-text-center" >Cuotas Restantes:
                     {{ prestamo._cuotasRestantes.toString() }} </ion-
                    label>
16            </ion-item>
17
18            <ion-item >
19              <ion-label class = "ion-text-center" >Cantidad Próxima
                    Cuota: {{ prestamo._cuotaMensual.toString() }} </ion
                    -label>
20            </ion-item>
21
22            <ion-item >
23              <ion-label class = "ion-text-center">Fin plazo acutal:
                    {{ prestamo._ultimoCheckeo }}</ion-label>
24            </ion-item>
25
26            <ion-item>
27              <ion-label class = "ion-text-center">Penalización por
                    Impago: {{ prestamo._penalizacionImpago.toString()
                    }}%</ion-label>
28            </ion-item>
29
30
31          </ion-list>
32          <ion-button (click)="pagarCuota(prestamo._direccion,
              prestamo._cuotaMensual)">PAGAR CUOTA ({{prestamo.
              _cuotaMensual.toString()}})</ion-button>
```

```
33          </ion-card-content>
34
35        </ion-card>
36      </ion-col>
```

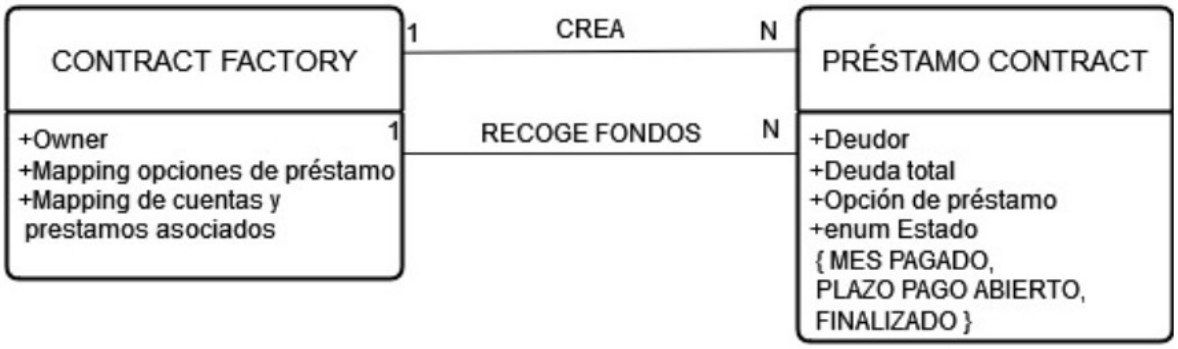
3.6.6. Registro del estado antes de realizar transferencias

En solidity es una buena práctica actualizar el estado de las variables antes de realizar transferencias de fondos, ya que esto ayuda a prevenir ataques en los que se reutiliza la misma transacción mas de una vez debido a la reentrada y garantiza que el estado del contrato refleje la situación actual antes de cualquier movimiento de fondos. Esto se puede ver en reflst:funcionpagarmensualidad donde se actualiza el estado del contrato antes de transferir los fondos al contrato *factory*

3.7. Diagrama Entidad - Relación de Smart Contracts

Aunque no sean entidades del sistema, para explicar detalladamente la arquitectura de de diseño se define otro diagrama entidad - relación en la figura 3.5:

Figura 3.5. Diagrama entidad relación que representa las relaciones y funciones entre los Smart contracts.



3.8. Descripción de Smart Contracts

3.8.1. Contract Factory

Tabla 3.15. Descripción de atributos del Contrato Contract Factory

Nombre	Tipo de dato	Descripción
Owner	Tipo Address	Dirección propietaria del sistema.
Tipos de préstamo	Mapping ((Uint =>Struct) "tipo prestamo")	Mapping que guarda los distintipos tipos de préstamo contratables por el usuario. Esta variable solo puede ser editada por la dirección de Owner.
TipoPréstamo	Struct	Conjunto de variables que definen un tipo de préstamo(Importe, intereses, deuda total, penalización de impago y cuotas)
Préstamos en curso	Mapping (Address =>Address[])	Mapping que relaciona el address de un usuario con las direcciones de sus préstamos en curso. Esta variable solo puede ser editada por la dirección de Owner.

Funciones:

Tabla 3.16. Descripción de funciones del Contrato Contract Factory

Nombre	Parámetro entrada	Parámetro salida	Descripcion
crearTipoPrestamo	Importe, interesTotal, penalización impago, número cuotas		Añade un nuevo tipo de préstamo al mapping tipos de préstamo.
crearPrestamo	Tipo de préstamo address de usuario		Añade un nuevo tipo de préstamo al mapping tipos de préstamo
chequeoMensual			Actualiza los estados de los contratos y calcula la nueva cuota mensual
consultarContratos		Mapping de préstamos en curso	Devuelve la listado de préstamos en curso
recuperarFondos	Cantidad		Transfiere unidades desde este contrato al Owner
invertirFondos	Cantidad		Transfiere unidades a este contrato.
consultarPrestamosUsuario	Dirección de usuario a consultar	Conjunto de de Direcciones de contrato de préstamo asociados a un mismo usuario	

3.8.2. Préstamo Contract

Tabla 3.17. Descripción de atributos del Contrato Préstamo Contract

Nombre	Tipo de dato	Descripcion
ID	Int	Identificador único
Estado	ENUMERADO	CUOTA PAGADA, PLAZO ABIERTO, FINALIZADO
CuotasRestantes	Int	Informa el tiempo límite acordado para devolver el préstamo
Prestatario	Tipo Address	Cuenta que ha contratado el préstamo
Deuda Restante	Float	Deuda restante del préstamo
Cuota Mensual	Float	Cantidad de la deuda a pagar mensualmente
Porcentaje penalización	Int	Porcentaje de penalización a la deuda en caso de impago

Funciones:

Tabla 3.18. Descripción de funciones del Contrato Préstamo Contract

Nombre	Parámetro entrada	Parámetro salida	Descripcion
pagoCuota			Transfiere al contrato la cuota previamente calculada, este importe es transferido directamente al contract factory después de dejar reflejado que se ha pagado la cuota.
aplicarPenalización		Nuevo valor de deuda	Añade el porcentaje de penalización del contrato a la deuda total
calculoCuotaMensua		Valor de la cuota mensual	A partir de la deuda restante y el numero de cuotas restantes calcula el la cuota.

3.9. Validación de resultados

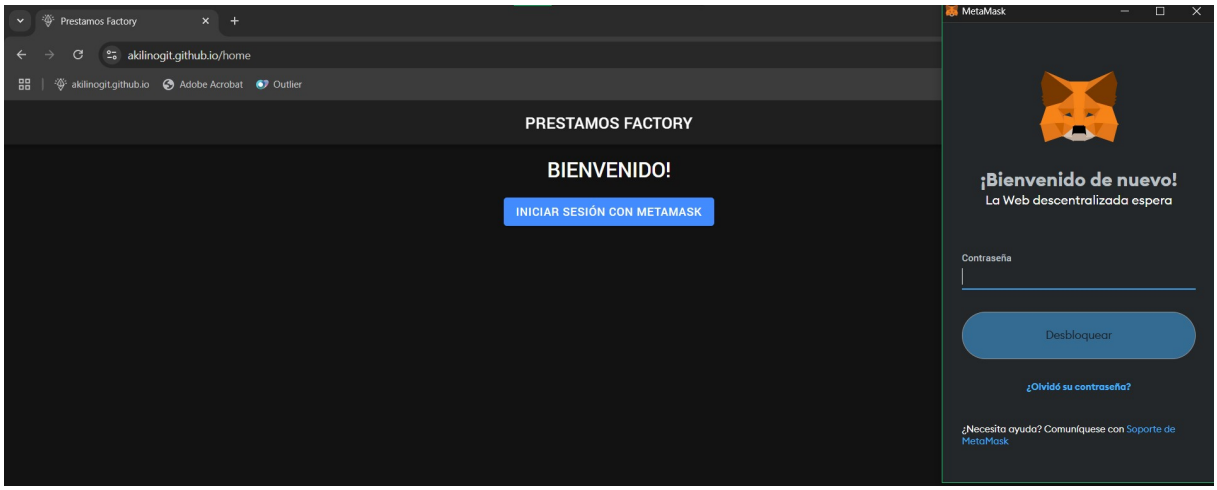
mocha Tras el desarrollo del sistema de préstamos descentralizado, es necesario comprobar si cumple con los objetivos planteados y cubre los casos de uso definidos. Para ello, se presentan pruebas mediante capturas de pantalla de la interfaz y registros en Etherscan, validando la ejecución de transacciones y el correcto funcionamiento de los contratos inteligentes en la blockchain de Ethereum.

El Objetivo principal (OP) 2 consiste en la unificación de las distintas funcionalidades del proyecto en una interfaz accesible a cualquier sujeto independiente de la formación. Los resultados se podrán ver a lo largo de la sección y se puedrá apreciar que la interfaz aunque si reúne todas las funcionalidades del sistema, no se ha recalcado en la accesibilidad e intuitividad. Durante el desarrollo del proyecto se ha decidido aportar más esfuerzo en la funcionalidad dejando la optimización de la interfaz de usuario para un posible trabajo futuro.

Para la validación de resultados del OP 1 se pueden evalúar los distintos objetivos secundarios:

1. **Objetivo secundario (OS) 1:** Una de las preocupaciones que puede suponer el desarrollo de un sistema de préstamos en una blockchain es la naturaleza anónima de este tipo de red. Por ello se ve en la figura 3.6 el uso de la plataforma Metamask como agente identificador externo para acceder a la aplicación.

Figura 3.6. Ejemplo de login con Metmask



2. La utilización de Metamask no solo nos sirve para delegar la identificación sino también para delegar en una empresa externa la información que identifica a nuestros clientes de forma que no hace falta grabarla en la red, esto nos ayuda con el el OS 5 el cual busca la optimización de información en la red para no revelar información sensible.

3. En el OS 2 se busca la eficiencia en la información y las funciones del sistema para evitar el alto consumo de energía a la hora de ejcutar las ordendes: Todas las funciones que tienen relación con la red son funciones muy atómicas y sencillas evitando los grandes procesamientos de datos en la red aunque es posible que la función que se encarga de comprobar los pagos de los contratos de forma mensual, para actualizar el estado o aplicar penalización, puede suponer una alta inversión por parte de la entidad financiera en caso de que tengo un volumen alto de préstamos, lo cual nos lleva a una posible decisión futura que sea la de limitar el número de préstamos en curso para mantener la rentabilidad el proyecto.

4. Haciendo referencia al OS 3 se adjuntan las siguientes imágenes del panel de control de la entidad financiera en el que se muestran 3 contratos de un usuario en curso antes y después de la comprobación mensual de los pagos de las cutoas. En ellas se pueden observar los distintos detalles:

   a) El balance de la entidad posterior al pago aumenta debido al corecto funcionamiento del pago

   b) Uno de los contratos no ha sido pagado y se le ha aplicado auomáticamente la penalización por impago

   c) Uno de los contratos correctamente pagado ha reducido su deuda restante y el número de cuotas restantes.

   d) En el caso de contrato que solo tenía una cuota restante se ha dejado de mostrar en la aplicación ya que no es necesario replicar más esta información pudiendose acceder mas adelante a ella ya que en la red todo queda grabado.

   Es importante para comprender las siguientes imagenes es importante conocer los siguientes conceptos:

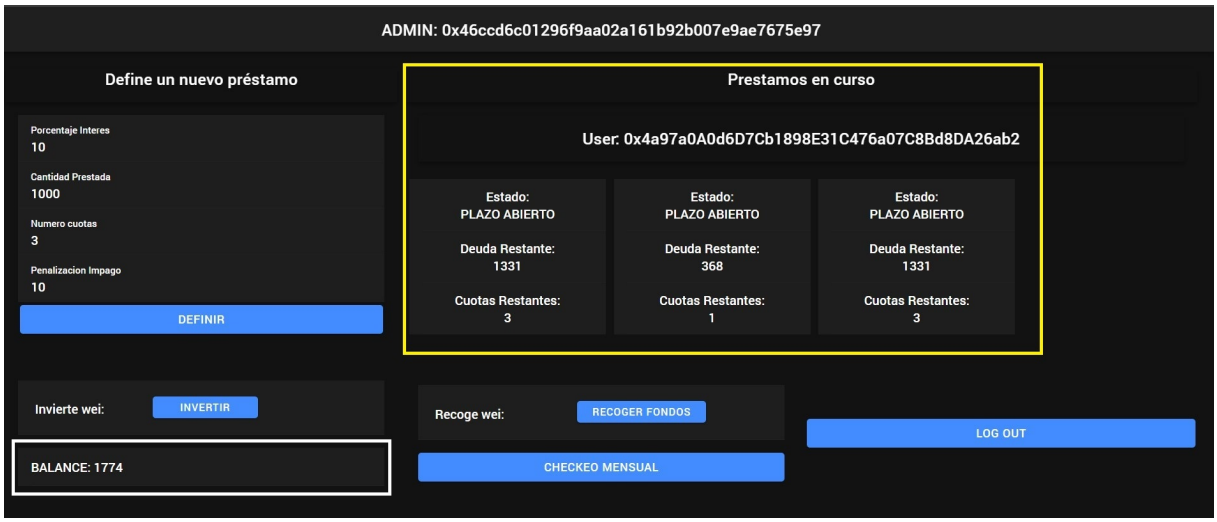
   a) **Wei:** En la red Ethereum es la unidad más pequeña de valor. Un Ether (ETH) está compuesto por un total de 10^18 wei.

   b) Las cadenas de caracteres que comienzan por `'0x'` en la red ethereum están reservadas como direcciones en la red que pueden apuntar a usuarios, entidades o Smart Contracts. Estas propiedades de nuestro sistema se pueden ver reflejadas en la figura 3.7 y tras la comprobación mensual del sistema en la figura 3.8

   Las últimas características declaradas junto con la posibilidad de que el dueño del sistema pueda recibir los wei del balance del sistema a través de la función Recoger Fondos"nos llevan al alcance del OS 6 el cuál busca encontrar la liquidez en este sistema. También es cierto que, de momento, este proyecto no cubre la opción en el que un usuario no pague sus cuotas indefinidamente aunque para ello se definió el Caso de Uso 12 en el que se define la posibilidad de añadir distintos tipos de colaterales como puede ser un NFT el cual sea transferido al sistema en caso de que el impago sea recurrente.

5. En el OS 5 se contempla la posibilidad de personalizar los contratos accesibles para distintos usuarios, esto, se puede clasificar como inalcanzable para este proyecto debido a la necesidad existente en procesar grandes cantidades de datos

Figura 3.7. Información de contratos previa a la comprobación mensual en el recuadro amarillo y balance de la entidad financiera en el recuadro blanco.



de clientes para poder crear un algoritmo y un sistema que cubra esta necesidad. De momento existe la posibildad de Definir nuevos préstamos contratables en el panel de control de la entidad financiera

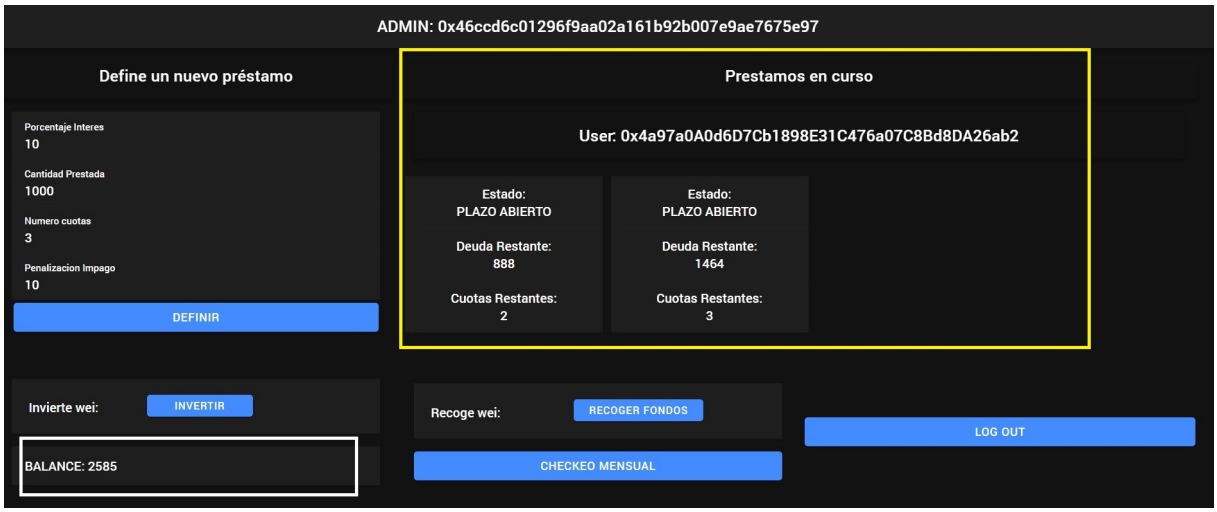
Para terminar de validar el OP 2 se adjunta captura de pantalla del panel de control del usuario en la figura 3.9.

Como última prueba de validación de este proyecto se realiza un ejemplo de contratación de préstamo adjuntado distintas capturas de la web Etherscan Sepolia en las que se puede comprobar distintos items que corroboran la veracidad del préstamo a través de la red. Sepolia.

Se realiza la petición de un prestamo por parte de un usuario con una cantidad de 1000 wei: **Contrato principal:** En la anterior figura 3.8 se puede comprobar que el balance del contrato principal es de 2585 wei. Después de la contratación del usuario, se puede ver a través de Etherscan Sepolia, y tal y como se muestra en la figura 3.10, que:

- El balance del contrato ha disminuido 1000 wei
- Se ha creado una nueva transacción relacionada con el contrato principal tipo 'Contract Creation' el cual es el contrato generado para el préstamo.
- Aunque debido al redondeo de la página aparezca otra transacción de 0 ETH, viendo el balance se puede comprobar que es de 1000 wei transferidos a la dirección

Figura 3.8. Información de contratos posterior a la comprobación mensual en el recuadro amarillo y balance de la entidad financiera en el recuadro blanco.



del usuario de la figura 3.9

Se adjunta también otra captura de pantalla en la figura 3.11 de la información del contrato generado con el pago de una cuota realizado cuyo valor coincide con las cuotas definidas del préstamo contratado. Con esta comprobación de función de pago en Etherscan Sepolia se da por validado que todas las funciones del proyecto se reflejan en la red otorgando veracidad al proyecto.

Figura 3.9. Panel de control usuario.

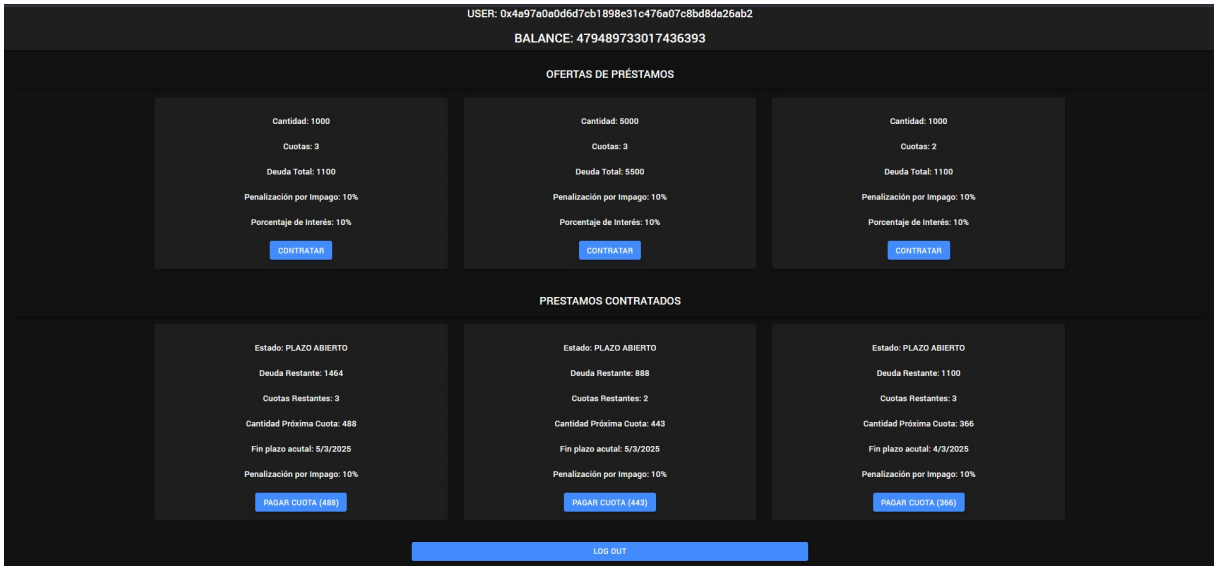


Figura 3.10. Información Etherscan del registro en la red SepoliaETH del SmartContract principal

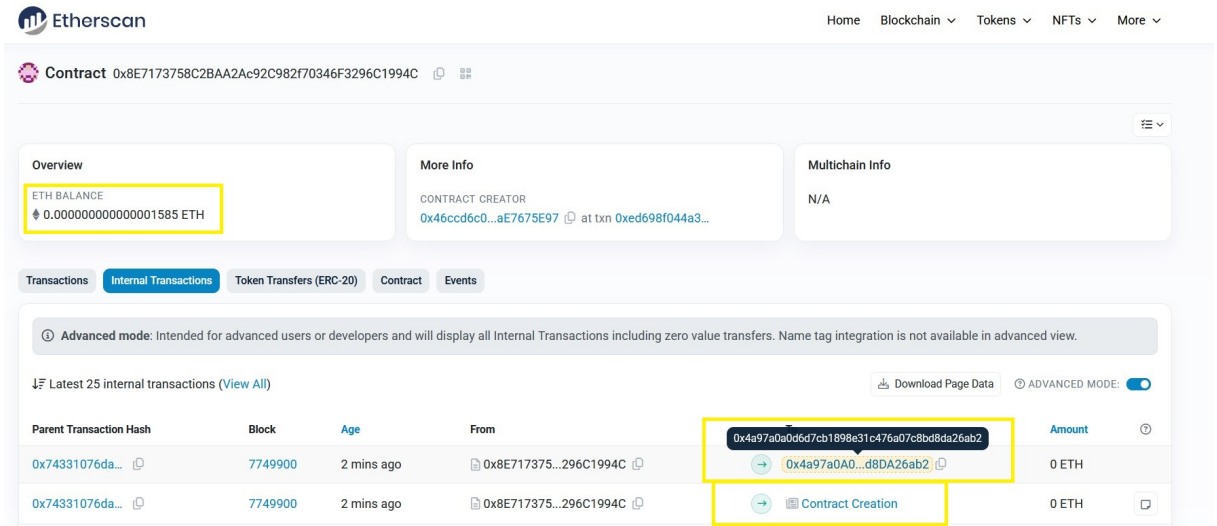
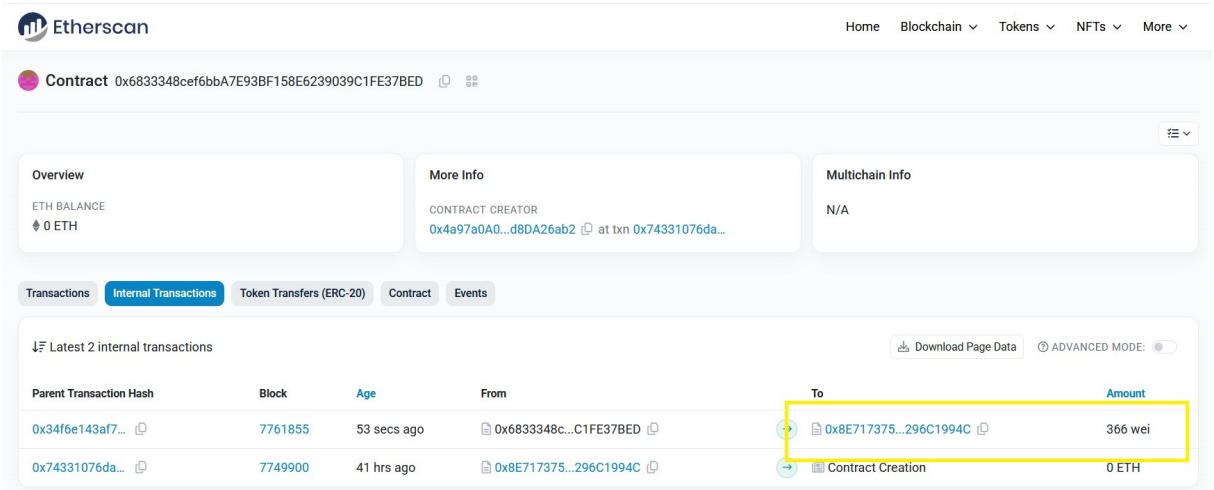


Figura 3.11. Registro en Sepolia en el que se puede ver una transacción o comprobante de pago Etherscan



4. Conclusiones

En este Proyecto Fin de Grado se ha desarrollado un sistema de préstamos descentralizado basado en la red Ethereum. Se puede afirmar que se ha obtenido una aplicación capaz de conectarse con una red blockchain y realizar transacciones en ella, y que gracias al registro de estas transacciones y cierta lógica aplicada a esta información, se puede componer un sistema de creación y control de préstamos digitales.

La interfaz de usuario realiza las conexiones necesarias con la red Blockchain y ordena las transacciones necesarias para cubrir las necesidades de creación y control de préstamos digitales.

Los Smart Contracts desplegados en la red ethereum definen correctamente, dentro de la red, la lógica aplicada a los atributos de las transacciones conformando los mecanismos necesarios para la funcionalidad del sistema.

Las características de seguridad e inmutabilidad que ofrecen las redes descentralizadas aplicadas a la información que el sistema guarda en la red Ethereum confirman la viabilidad técnica de las operaciones financieras de este sistema.

Durante la implementación y validación, se ha demostrado el correcto funcionamiento del sistema mediante pruebas en redes de desarrollo.

Estos resultados confirman que la solución desarrollada cumple con los objetivos iniciales los cuales intentaban crear un sistema capaz de utilizar la tecnología disponible en la actualidad para construir un sistema sólido, confiable, transparente y autónomo para la gestión de préstamos respaldados por criptomonedas. Acercando esta tecnología al usuario común y reuniendo las acciones necesarias en una única pantalla de control.

Finalmente, se concluye que la adopción de tecnologías DeFi y contratos inteligentes tiene el potencial de transformar el acceso a servicios financieros, reduciendo costos y aumentando la inclusión financiera a nivel global. Sin embargo, persisten desafíos como la volatilidad del valor de las criptomonedas y los costos de transacción en Ethereum.

4.1. Impactos sociales y ambientales del desarrollo de este sistema a gran escala

La adopción masiva de un sistema de préstamos descentralizado basado en blockchain y contratos inteligentes podría generar importantes transformaciones en el ámbito financiero y social. La eliminación de intermediarios permitiría un acceso más democrático a los servicios financieros, especialmente en regiones con sistemas bancarios limitados o restrictivos. Sin embargo, este modelo también presenta retos significativos en términos de inclusión digital, regulación y sostenibilidad ambiental.

4.1.1. Impactos sociales

1. La posibilidad de otorgar préstamos sin la necesidad de bancos tradicionales podría beneficiar a millones de personas no bancarizadas en economías emergentes. Sin embargo, el acceso a estos servicios dependería del conocimiento en el uso de criptomonedas, wallets digitales y plataformas DeFi, lo que podría generar una brecha digital entre quienes dominan estas herramientas y quienes no.

2. La descentralización del sistema podría entrar en conflicto con regulaciones financieras existentes. Gobiernos y bancos centrales podrían imponer restricciones al uso de plataformas DeFi, especialmente en países donde el control financiero es una herramienta de política económica. Además, la anonimidad en las transacciones podría plantear problemas relacionados con el financiamiento ilícito si no se establecen mecanismos de verificación de identidad adecuados.

3. La automatización de los préstamos a través de contratos inteligentes podría reducir la demanda de intermediarios financieros, afectando a trabajadores del sector bancario y generando resistencia por parte de instituciones tradicionales. Sin embargo, también abriría oportunidades en nuevas áreas tecnológicas, creando demanda de profesionales en blockchain, ciberseguridad y programación de contratos inteligentes.

4.1.2. Impactos ambientales

1. Ethereum, aunque ha reducido su impacto ambiental tras su transición de Proof of Work (PoW) a Proof of Stake (PoS), sigue dependiendo de una infraestructura global de nodos y validadores que consumen energía. A gran escala, este sistema de préstamos podría contribuir al aumento de la demanda energética de los centros de datos y redes descentralizadas, lo que requeriría soluciones de eficiencia energética y compensación de emisiones.
2. La minería y validación de transacciones en redes blockchain han demostrado afectar la disponibilidad de tarjetas gráficas (GPUs), lo que en el pasado ha impactado negativamente industrias como la inteligencia artificial, videojuegos y computación científica. Si el crecimiento del sistema de préstamos descentralizado aumentara la demanda de hardware, podría contribuir a problemas de escasez de equipos electrónicos.

Si bien un sistema de préstamos descentralizado tiene el potencial de democratizar el acceso financiero y reducir costos, también presenta desafíos sociales y ambientales que deben abordarse antes de una implementación a gran escala. Para mitigar estos impactos, sería fundamental desarrollar estrategias de educación digital, establecer mecanismos de regulación equilibrados y mejorar la eficiencia energética del sistema a través de blockchains más sostenibles.

Solo con un enfoque responsable y sostenible se podrá garantizar que este tipo de innovación beneficie a la sociedad sin comprometer el bienestar ambiental.

4.1.3. Objetivos de desarrollo sostenible. Agenda 2030

La Agenda 2030 para el Desarrollo Sostenible es un plan global adoptado en 2015 por los Estados Miembros de la ONU con el objetivo de erradicar la pobreza, proteger el planeta y garantizar la prosperidad para todas las personas. Para lograrlo, se establecieron 17 Objetivos de Desarrollo Sostenible (ODS), que abarcan áreas clave como la educación, la igualdad de género, la acción climática, la innovación y el crecimiento económico, entre otras.

Estos objetivos buscan equilibrar el desarrollo social, económico y ambiental, promoviendo un modelo sostenible que asegure el bienestar de las generaciones actuales y

futuras. Además, llaman a la colaboración entre gobiernos, empresas y la sociedad civil para alcanzar metas concretas antes del año 2030.

En la figura 4.1 se pueden ver estos objetivos:

Figura 4.1. Objetivos de desarrollo sostenible de la agenda europea para 2030



Se puede concluir que este proyecto tiene un gran potential de influir positivamente en los objetivos relacionados en el ámbito financiero como los ODS 8 y 9 y también en el ámbito social podría promover los ODS 10 y 16 aunque también es importante destacar que para que proyectos de este estilo aporten positivamente estos ODS habría que invertir mucho en educación y crear estructuras sólidas en el sistema financiero actual.

En el apartado ambiental, aunque no esta todo perdido y se pueden estudiar maneras de reducir la energía gastada en la computación de esta tecnología, todo apunta a que esta clase de proyectos van en contra de los objetivos climáticos como los ODS 11, 12 y 13.

Referencias

[1] E. Dans, «*Planteando el bitcoin como alternativa monetaria*», 2024. dirección: https://www.enriquedans.com/2024/12/planteando-el-bitcoin-como-alternativa-monetaria.html

[2] A. P. Sanz, *Repositorio de este proyecto: Sistema de préstamos descentralizado basado en la red Blockchain de Ethereum*, 2024. dirección: https://github.com/AkilinoGit/akilinogit.github.io

[3] J. L. M. T. CARLOS DOLADER RETAMAL JOAN BEL ROIG, «*LA BLOCKCHAIN: FUNDAMENTOS, APLICACIONES Y RELACIÓN CON OTRAS TECNOLOGÍAS DISRUPTIVAS*», 2017. dirección: https://www.mintur.gob.es/Publicaciones/Publicacionesperiodicas/EconomiaIndustrial/RevistaEconomiaIndustrial/405/DOLADER%2C%20BEL%20Y%20MU%C3%91OZ.pdf

[4] M. U. G. BARTOLOMEO Alejandro, «*INTRODUCCIÓN A LA TECNOLOGÍA BLOCKCHAIN: SU IMPACTO EN LAS CIENCIAS ECONÓMICAS S*», 2020. dirección: https://bdigital.uncuyo.edu.ar/objetos_digitales/15304/14.-introduccinalatecnologia.pdf

[5] Neocheck, *SepoliaFaucet*, 2019. dirección: https://www.neocheck.es/wp-content/uploads/2019/09/funcionamiento-blockchain-permisionada-800x416.png

[6] Amazon, «*¿Qué es la tecnología de cadena de bloques?*» -. dirección: https://aws.amazon.com/what-is/blockchain/?aws-products-all.sort-by=item.additionalFields.productNameLowercase&aws-products-all.sort-order=ascf

[7] Wikipedia, «*Ethereum*», -. dirección: https://es.wikipedia.org/wiki/Ethereum

[8] Solana, *Solana*, -. dirección: https://solana.com/es

[9] BNBChain, *BNBChain*, -. dirección: https://www.bnbchain.org/es-LA

[10] Sepolia, *Sepolia*, -. dirección: https://sepolia.etherscan.io/

[11] SepoliaFaucet, *SepoliaFaucet*, -. dirección: https://www.alchemy.com/faucets/ethereum-sepolia

[12] Coinbase, *Coinbase*, -. dirección: https://www.coinbase.com/es-es/wallet

[13] RabbyWallet, *RabbyWallet*, -. dirección: https://rabby.io/

[14] MetaMask, *MetaMask*, -. dirección: https://metamask.io/

[15] Megaacademi.io, *MetaMask – ¿Qué es MetaMask y qué ventajas tiene?* -. dirección: https://megaacademy.io/metamask/

[16] Solidity, *Solidity*, -. dirección: https://soliditylang.org/

[17] RemixIDE, *RemixIDE*, -. dirección: https://remix-project.org/?lang=es

[18] NodeJS, *NodeJS*, -. dirección: https://nodejs.org/es

[19] Truffle, *Truffle*, -. dirección: https://archive.trufflesuite.com/

[20] Ganache, *Ganache*, -. dirección: https://archive.trufflesuite.com/ganache/

[21] EtherScan, *EtherScan*, -. dirección: https://etherscan.io/

[22] U. Europea, *Reglamento Europeo sobre los Criptoativos*, 2023. dirección: https://eur-lex.europa.eu/ES/legal-content/summary/european-crypto-assets-regulation-mica.html

Índice de términos

Glosario

App Store Tienda digital de aplicaciones.. 11

Bitcoin Red blockchain original básada únicamente en transacciones y la que mayor valor ha conservado. 10

Blockchain Blockchain es una tecnología de registro distribuido que almacena datos en bloques encadenados de forma segura, inmutable y descentralizada. Se usa principalmente para transacciones digitales, como criptomonedas, pero también tiene aplicaciones en contratos inteligentes, logística y más.. 3–5

dApp Descentrelized App o aplicación descentralizada, es una aplicación que funciona sobre una red descentralizada de blockchain en lugar de un servidor central. Utiliza contratos inteligentes para gestionar su lógica y datos. 4, 5, 11, 13, 15, 22

DeFi Descentrelized Finance: Sistema financiero en un entorno descentralizdo, como puede ser una red blockchain. 1–5, 10–12, 45

ERC-20 estándar técnico para crear y manejar tokens en la blockchain de Ethereum. Define reglas básicas que los tokens deben seguir, como transferencias, balance y aprobación, lo que garantiza compatibilidad con contratos inteligentes y aplicaciones descentralizadas.. 11

ETH Ether, es la criptomoneda de la red Ethereum. 1, 11, 12, 17, 52

Ether Ether (ETH) es la criptomoneda nativa de la blockchain de Ethereum. Se utiliza para pagar tarifas de transacción (gas) y ejecutar contratos inteligentes dentro de la red. También actúa como un activo digital que puede ser intercambiado o utilizado en aplicaciones descentralizadas (DApps) .. 40

Ethereum Red blockchain cuya moneda principal es el ETH y es capaz de ejecutar Smart Contract. 3, 10, 13, 30, 51

Euribor Es el tipo de interés promedio al que los principales bancos de la zona euro se prestan dinero entre sí en el mercado interbancario. Se calcula diariamente y sirve como referencia para numerosos productos financieros, especialmente para hipotecas a tipo variable en Europa. . 22

Framework Conjunto de herramientas, bibliotecas y convenciones diseñadas para facilitar el desarrollo de aplicaciones, proporcionando una estructura base que simplifica y organiza el código. 6, 13

Gas Cantidad de ETH necesario para la ejecución de una transacción. Este valor varía. . 3, 11, 18

hash Un hash es un valor único generado por una función hash a partir de un conjunto de datos de entrada. Se utiliza para representar esos datos de manera compacta y segura.. 6, 7

hashing Proceso de realizar un hash. 9

Javascript JavaScript es un lenguaje de programación interpretado, ligero y versátil, utilizado principalmente para crear contenido web interactivo.. 13, 14

Metamask Metamask es una extensión de navegador que permite a los usuarios interactuar con aplicaciones descentralizadas (DApps) en la red Ethereum, facilitando la gestión de cuentas y la firma de transacciones.. 31

NFT Non Fungible Token, activo digital único que representa propiedad o autenticidad de un objeto, como arte, música o coleccionables almacenado en una blockchain. A diferencia de las criptomonedas 'no fungible' significa que no es divisible. 11, 12, 22, 40

script Conjunto de instrucciones o código escrito en un lenguaje de programación o de scripting que se ejecuta automáticamente para realizar tareas específicas.. 10

SHA-256 Secure Hash Algorithm 256, es una función hash criptográfica que genera un valor único de 256 bits (32 bytes) a partir de una entrada. 9

Smart Contract Contrato inteligente: Script de pgrogramación que es capaz de automatizar procesos y transacciones dentro de una red blockchain programable como es la de Ethereum. 3, 11, 51, 53

Smart Contract Programa o código autoejecutable que se ejecuta en una blockchain. Define reglas y condiciones preestablecidas y, cuando se cumplen, ejecuta automáticamente acciones sin necesidad de intermediarios. Se usa en transacciones, acuerdos digitales y aplicaciones descentralizadas.. 4, 5, 10, 13, 14, 22, 24, 25, 30, 40, 45, III

Solidity Lenguaje de programación utilizado para programarSmart Contract. 11, 24

Tipo Address En solidity existe el tipo 'address' utilizado para distinguir que valores corresponden a el de una dirección en la red.. 35, 37

UI User Interface: Interfaz de usuario. 6

Web3 Web3 es la evolución de la web que utiliza tecnología blockchain para descentralizar aplicaciones y servicios, permitiendo mayor control, privacidad y propiedad de los datos por parte de los usuarios.. 4, 5, 12

Wei Wei es la unidad más pequeña de Ether (ETH), donde 1 ETH = 1,000,000,000,000,000,000 Wei (10^{18} Wei). Es la unidad de medida utilizada en las transacciones de la blockchain de Ethereum.. 31

Siglas

DNI Documento Naciona de Identidad. 12

EVM Ethereum Virtual Machine, Máquina Virtual de Ethereum. 11, 12, 14

ODS Objetivos de Desarrollo Sostenible. 47, 48

OP Objetivo principal. 38, 39, 41

OS Objetivo secundario. 39, 40

PFG proyecto de fin de grado. 4

PoS Proof of stake, Prueba de participación. 8, 11, 47

PoW Proof of Work, Prueba de trabajo. 8, 47

